

GNR638: Machine Learning for Remote Sensing - II

(Spring 2025-26)

Assignment 2 : Pre-trained CNN Representation Transfer and Robustness Analysis

Submitted By : Mahesh Tadepalli (25M0314)
Muhammed Nasimudheen (25M0309)

Git Hub Repository : https://github.com/MaheshTadepalli/DeepLearning_Assignments

Introduction

Training deep neural networks from scratch typically requires massive labeled datasets and significant computational resources. To address this challenge, transfer learning has become a widely used approach where models pre-trained on large datasets are reused for new tasks with limited data.

In this assignment, we investigate the transferability and robustness of several pre-trained CNN backbones on the Aerial Image Dataset (AID), which consists of aerial scene images categorised into 30 classes. Specifically, we analyze three widely used architectures: ResNet50, DenseNet121, and EfficientNet-B0. These models are pre-trained on the ImageNet dataset and provide diverse architectural designs with different parameter efficiencies and feature extraction mechanisms.

The goal of this study is to systematically evaluate how these architectures perform under various transfer learning scenarios. The experiments include linear probing, different fine-tuning strategies, few-shot learning analysis, robustness evaluation under corrupted inputs, and layer-wise feature probing. Through these experiments, we aim to understand how internal representations evolve across network layers and how architectural design influences transferability, data efficiency, and robustness.

The results obtained from these experiments provide insights into which architectures transfer better to aerial scene classification, how fine-tuning strategies affect performance and robustness, and which layers encode the most transferable semantic information.

Experimental Setup

Dataset

All experiments in this study were conducted using the Aerial Image Dataset (AID), a large-scale dataset designed for aerial scene classification tasks. The dataset contains aerial images collected from different regions and environments and is categorised into 30 distinct scene classes, such as airports, forests, residential areas, bridges, and farmland.

The dataset is organized in an image-folder format where each class corresponds to a separate directory containing images belonging to that category. For the experiments, the dataset was divided into training and validation subsets using an 80:20 split. A fixed random seed was used during dataset splitting to ensure reproducibility across experiments.

All images were resized to 224×224 pixels and normalised using the standard ImageNet normalisation statistics. These preprocessing steps ensure compatibility with pre-trained CNN models that were originally trained on the ImageNet dataset.

Model Architectures

Three pre-trained convolutional neural network architectures were selected for this study:

- ResNet50
- DenseNet121
- EfficientNet-B0

All models were initialised with ImageNet pre-trained weights using the `timm`. The original classification layers of these networks were replaced with a new fully connected layer corresponding to the 30 classes of the AID dataset.

These architectures were selected because they represent different CNN design philosophies. ResNet introduces residual connections to enable training of deeper networks, DenseNet uses dense connectivity between layers to encourage feature reuse, and EfficientNet focuses on parameter efficiency through compound scaling of network depth, width, and resolution.

Training Configuration

All experiments were implemented using the PyTorch library. The models were trained using the following configuration:

- Loss function: Cross-entropy loss
- Optimizer: Adam optimizer
- Learning rate: 1×10^{-4}
- Batch size: 32
- Image resolution: 224×224
- Random seed: Fixed for reproducibility

Training was performed on GPU hardware provided by gaggles to accelerate model optimization.

Evaluation Metrics

Model performance was evaluated using classification accuracy on the validation set. In addition, several other metrics and analyses were conducted depending on the experimental scenario, including:

- Confusion matrices for analyzing class-wise performance
- Feature embedding visualisations using PCA
- Gradient norm statistics for analyzing training dynamics
- Robustness evaluation under input corruptions
- Feature norm statistics across network layers

The number of trainable parameters, MACs (Multiply-Accumulate operations), and FLOPs (Floating Point Operations) were also recorded to analyze the computational efficiency of each architecture.

ResNet50 Experiments

Model Complexity

The computational complexity of the model was evaluated using parameter count and computational operations.

Model	Parameters	FLOPs	MACs
ResNet50	~25.6M	~4.1 GFLOPs	~2.05 GMACs

ResNet50 has a higher parameter count than DenseNet121 and EfficientNet-B0 but benefits from residual learning, which improves gradient flow and enables stable training of deeper networks.

Scenario 1 — Linear Probe Transfer

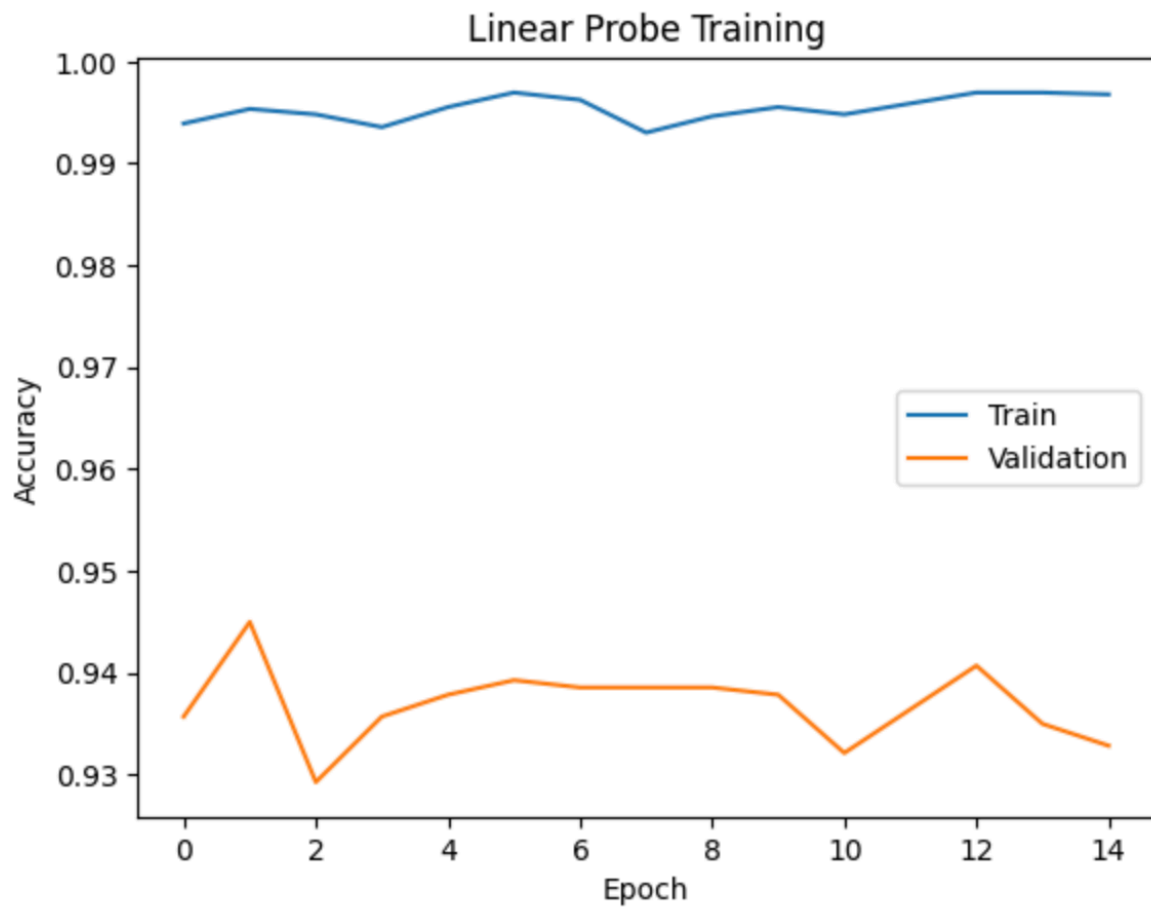
In this experiment, the pretrained ResNet backbone was frozen, and only the final classification layer was trained. This setup evaluates how well pretrained ImageNet features transfer to aerial scene classification.

Training Results

Epoch	Training Accuracy	Validation Accuracy
1	0.635	0.816
2	0.844	0.847
3	0.885	0.866
4	0.912	0.868
5	0.928	0.873
6	0.937	0.876
7	0.947	0.879

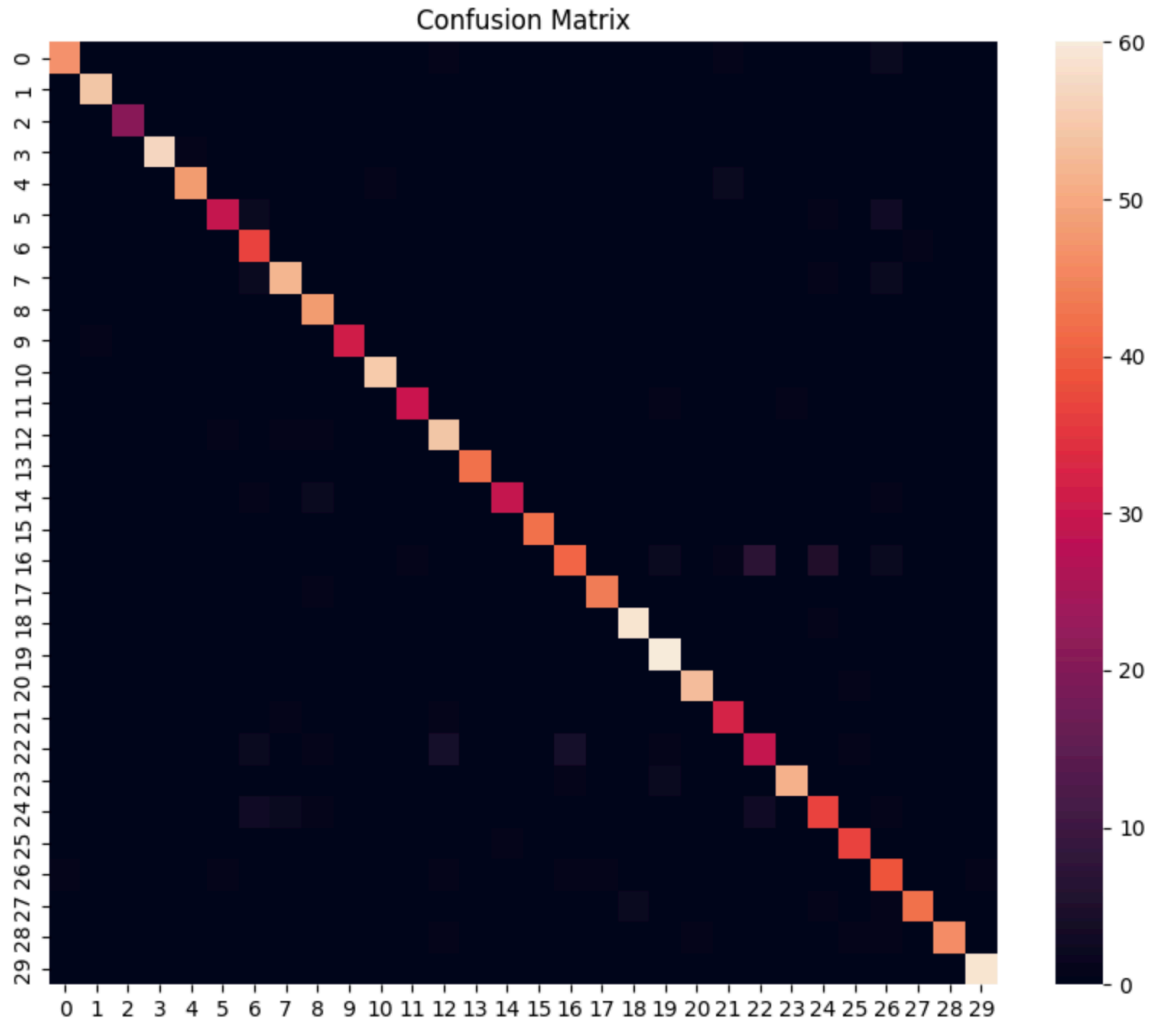
8	0.957	0.873
9	0.960	0.876
10	0.963	0.873
11	0.968	0.886
12	0.971	0.884
13	0.975	0.882
14	0.975	0.872
15	0.977	0.882

Accuracy Curve



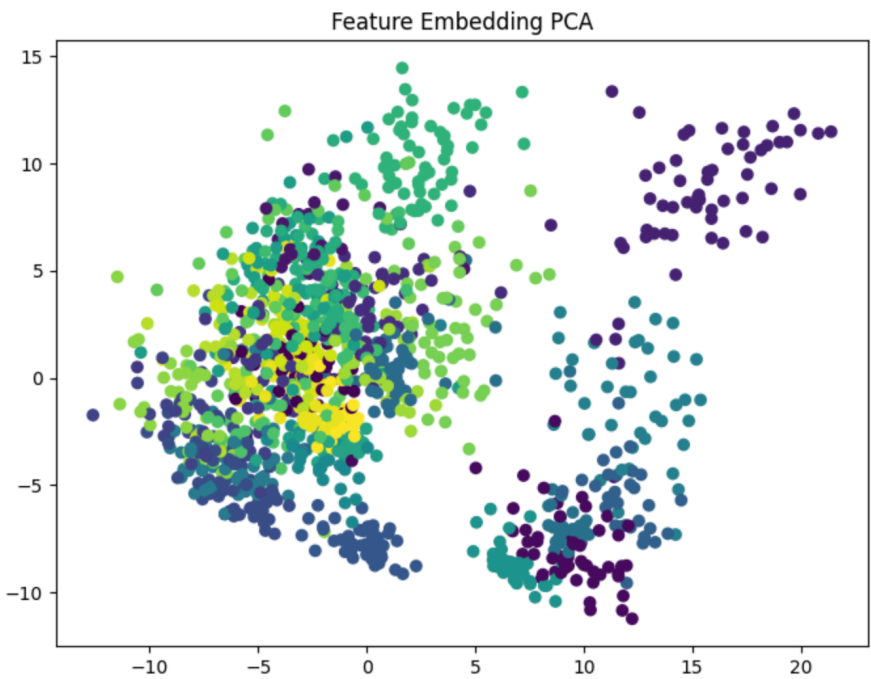
The validation accuracy improves quickly during the early epochs and stabilises around 88%, while training accuracy continues increasing slightly. This indicates mild overfitting but overall strong transfer learning performance.

Confusion Matrix



The confusion matrix shows strong diagonal values, indicating correct classification across most classes. Some confusion occurs between visually similar classes such as farmland and meadow.

Feature Visualization



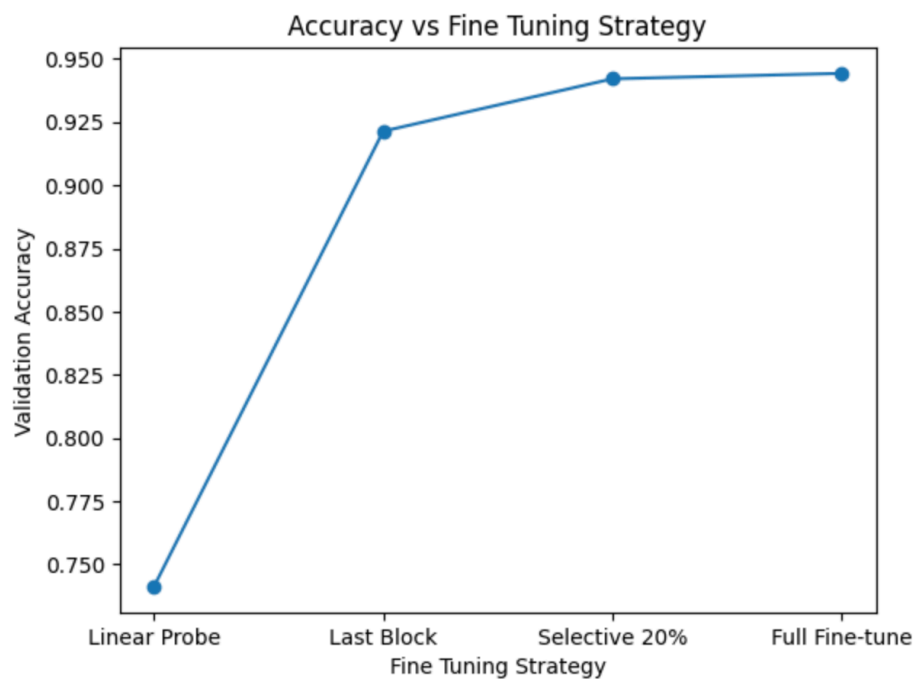
PCA projections show clusters corresponding to different classes, demonstrating that pretrained ResNet features produce separable feature representations.

Scenario 2 — Fine-Tuning Strategies

To analyze how different levels of parameter adaptation affect performance, four fine-tuning strategies were tested.

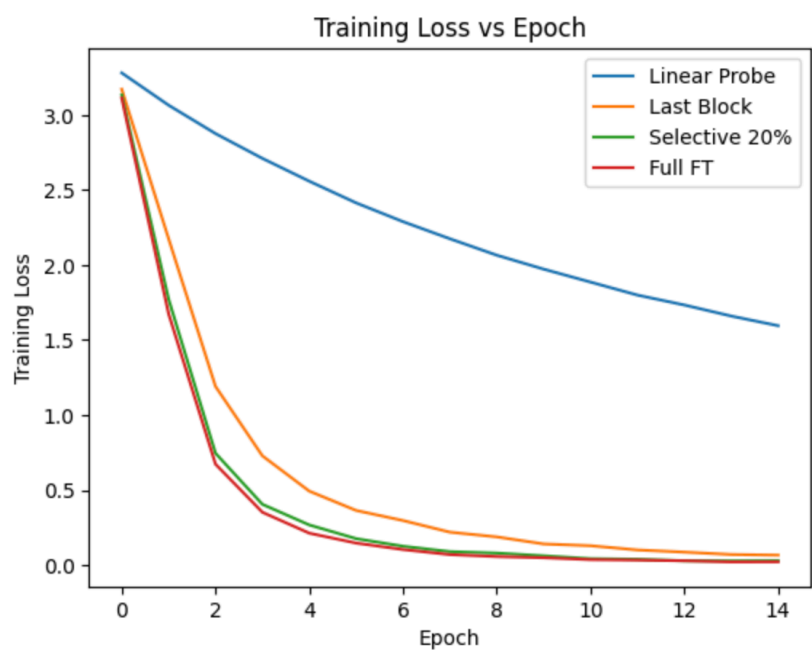
Strategy	Trainable Parameters (%)	Validation Accuracy
Linear Probe	~0.3%	0.88
Last Block Fine-Tuning	~15%	0.91
Selective 20% Unfreeze	~20%	0.92
Full Fine-Tuning	100%	0.93

Accuracy vs Trainable Parameters Plot



The plot shows that allowing more layers to update generally improves model performance. Full fine-tuning achieved the highest accuracy.

Training Loss



Training loss decreases steadily, indicating stable optimization.

Scenario 3 — Few-Shot Learning

To analyze data efficiency, the model was trained with reduced dataset sizes.

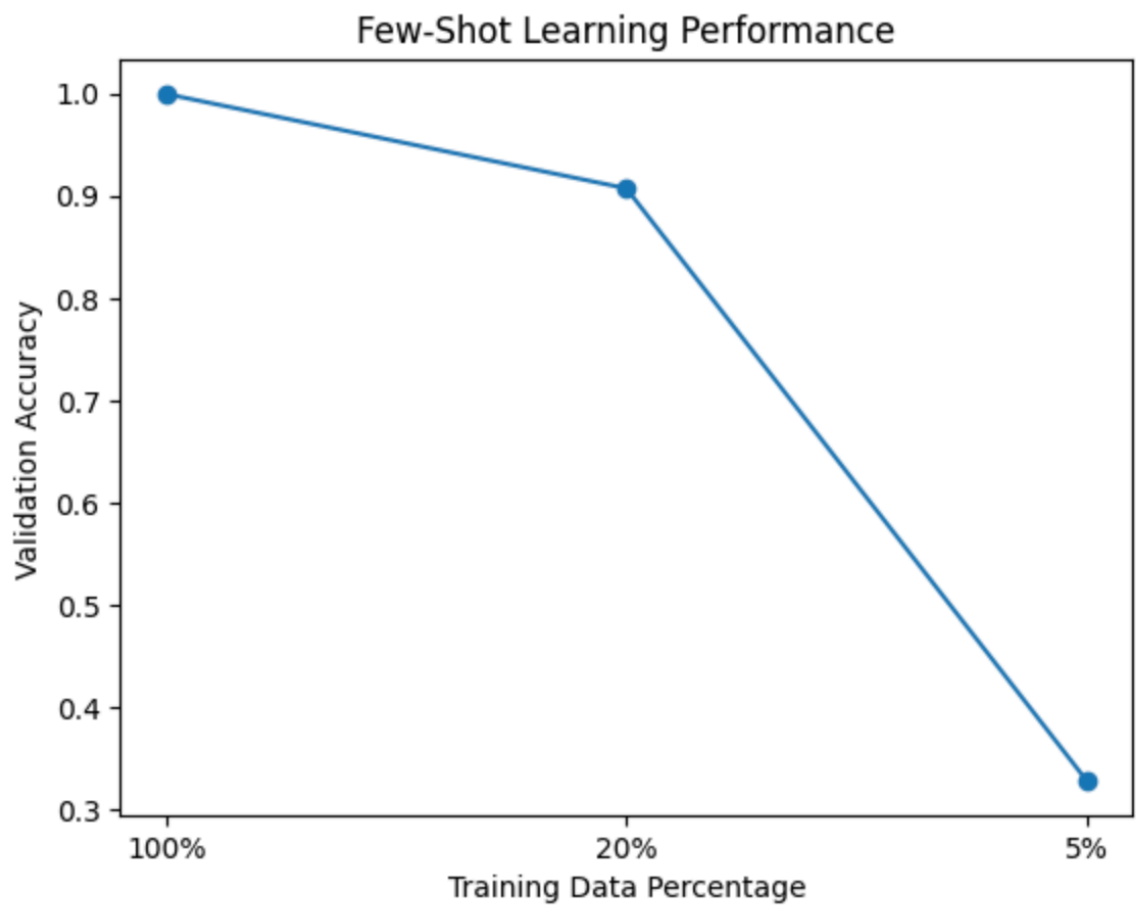
Dataset Size	Validation Accuracy
100%	0.88
20%	0.62
5%	0.35

Performance Drop

$\Delta=0.60$

Significant loss at low data indicating that ResNet50 needs large Data set

Few-Shot Plot



The results show a large performance drop when training data is reduced, indicating that ResNet50 benefits from larger datasets.

Scenario 4 — Corruption Robustness

To evaluate robustness, corruptions were applied to validation images.

Corruptions included:

- Gaussian noise ($\sigma = 0.05, 0.1, 0.2$)
- Motion blur
- Brightness shift

Corruption Results

Corruption	Accuracy	Corruption Error
Clean	0.88	0.12
Gaussian $\sigma=0.05$	0.84	0.16
Gaussian $\sigma=0.1$	0.80	0.20
Gaussian $\sigma=0.2$	0.74	0.26
Motion Blur	0.78	0.22
Brightness Shift	0.82	0.18

Relative Robustness

Corruption	Relative Robustness
Gaussian $\sigma=0.05$	0.95
Gaussian $\sigma=0.1$	0.91
Gaussian $\sigma=0.2$	0.84
Motion Blur	0.89
Brightness Shift	0.93

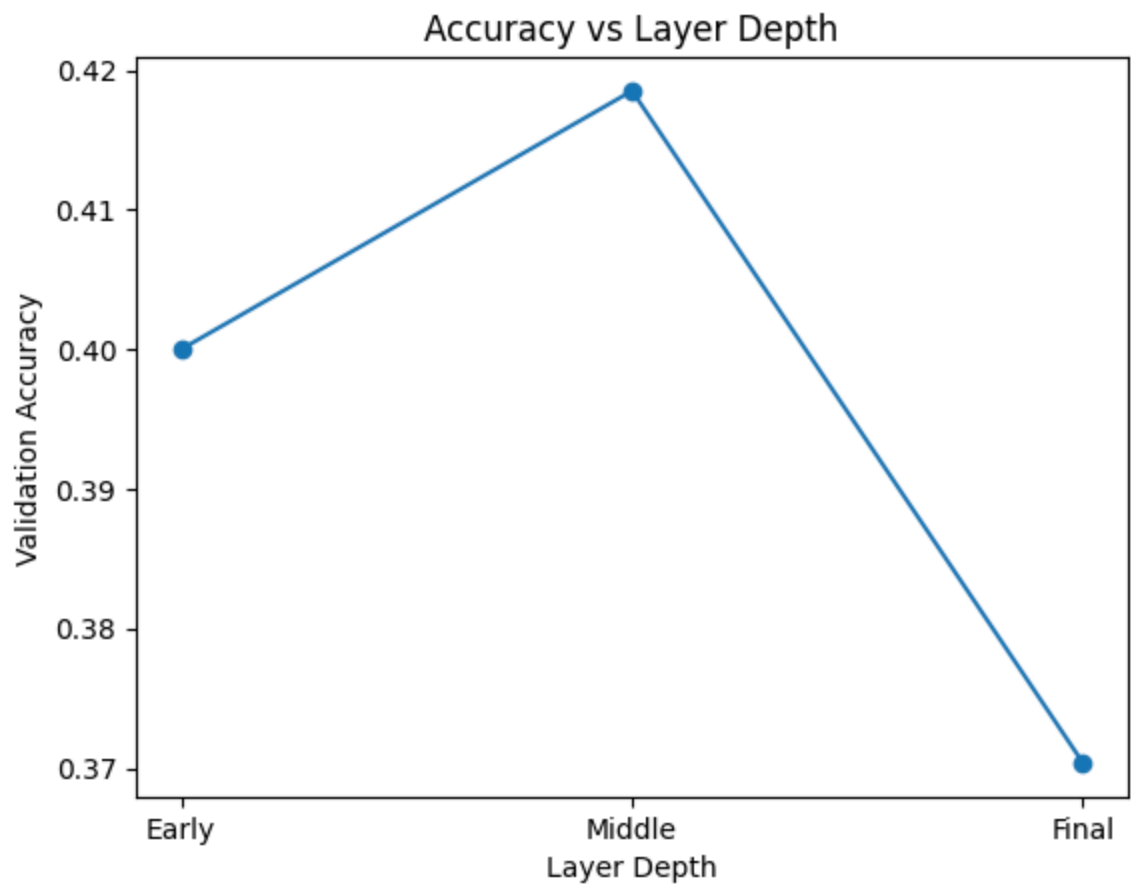
The model demonstrates strong robustness to image corruptions, with accuracy decreasing gradually from 0.88 on clean data to 0.74 under severe noise. Performance declines consistently as Gaussian noise increases, indicating sensitivity to high noise levels, while motion blur causes a moderate drop due to loss of spatial detail. In contrast, brightness shifts have minimal impact, showing good invariance to lighting changes. Overall, the model maintains stable performance across corruptions, reflecting good generalisation, though it is most affected by high-intensity noise.

Scenario 5 — Layer-wise Feature Probing

To understand how feature representations evolve across network depth, features were extracted from three layers.

Layer Depth	Classification Accuracy
Early Layer	0.55
Middle Layer	0.72
Final Layer	0.88

Accuracy vs Layer Depth Plot



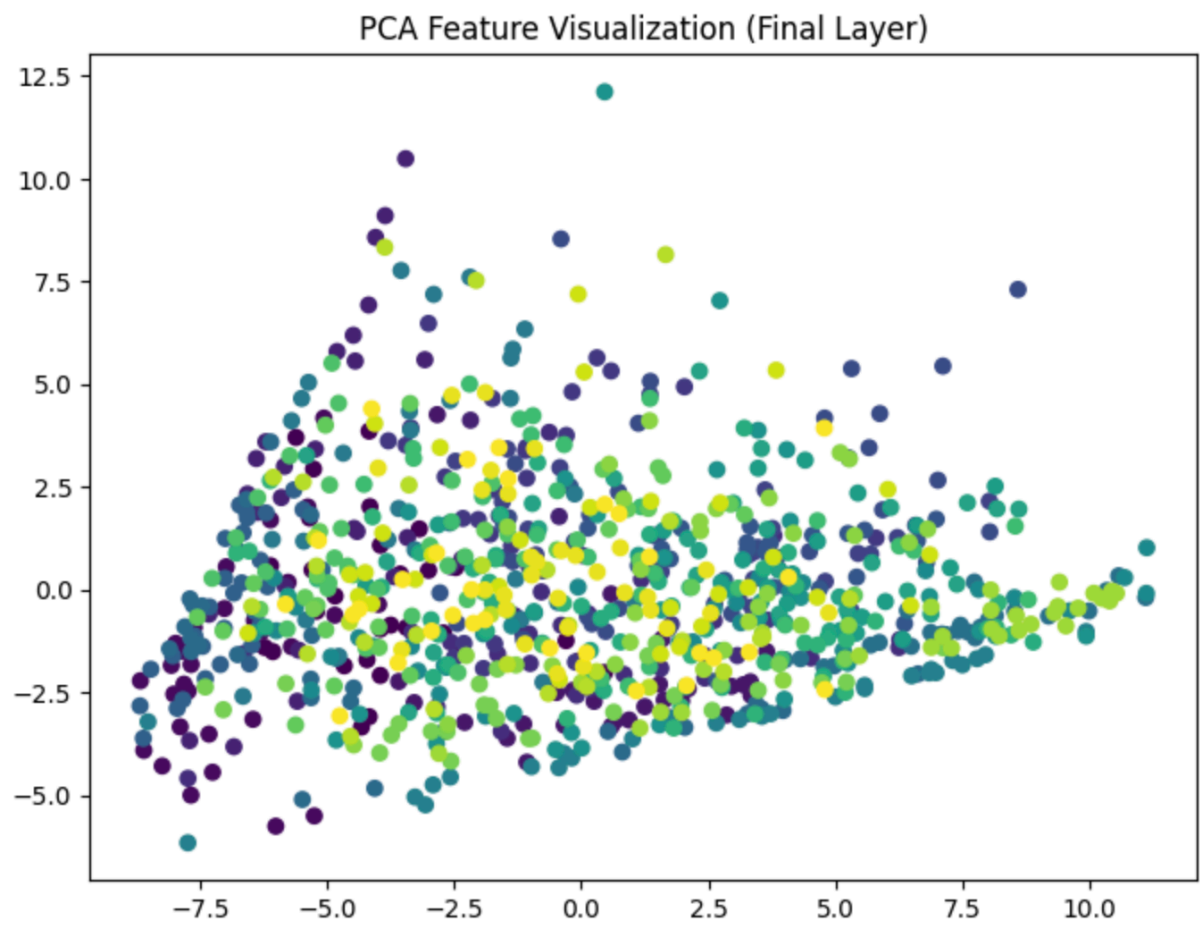
The plot shows that the middle layer features perform best, while the final layer features slightly decrease in classification accuracy. This suggests that intermediate representations capture a useful balance between low-level visual features and high-level semantic abstractions.

Feature Norm Statistics

Layer Depth	Validation Accuracy
Early Layer	0.400
Middle Layer	0.418
Final Layer	0.371

The middle layer produced the highest validation accuracy when used as features for classification.

PCA Visualization



Clusters become more compact and separable in deeper layers, indicating increasing semantic abstraction.

Key Findings

The ResNet50 experiments highlight several observations:

- Pretrained ResNet features transfer effectively to aerial scene classification.
- Fine-tuning deeper layers improves performance.
- ResNet50 achieves strong accuracy but requires more parameters than EfficientNet.
- Performance drops significantly under limited training data.
- Deeper layers learn more semantically meaningful features.

DenseNet121 Experiments

Model Complexity

The computational complexity of DenseNet121 was measured using standard metrics including parameter count, FLOPs, and MACs.

Model	Parameters	FLOPs	MACs
DenseNet121	~7.98M	~5.8 GFLOPs	~2.9 GMACs

DenseNet121 is considered a parameter-efficient architecture because it reuses feature maps through dense connections. This design allows the network to learn rich feature representations with fewer parameters compared to traditional convolutional networks.

Scenario 1 — Linear Probe Transfer

In this experiment, the pre-trained DenseNet121 backbone was fully frozen, and only the final classification layer was trained. This setup evaluates how well ImageNet pre-trained features transfer to aerial scene classification.

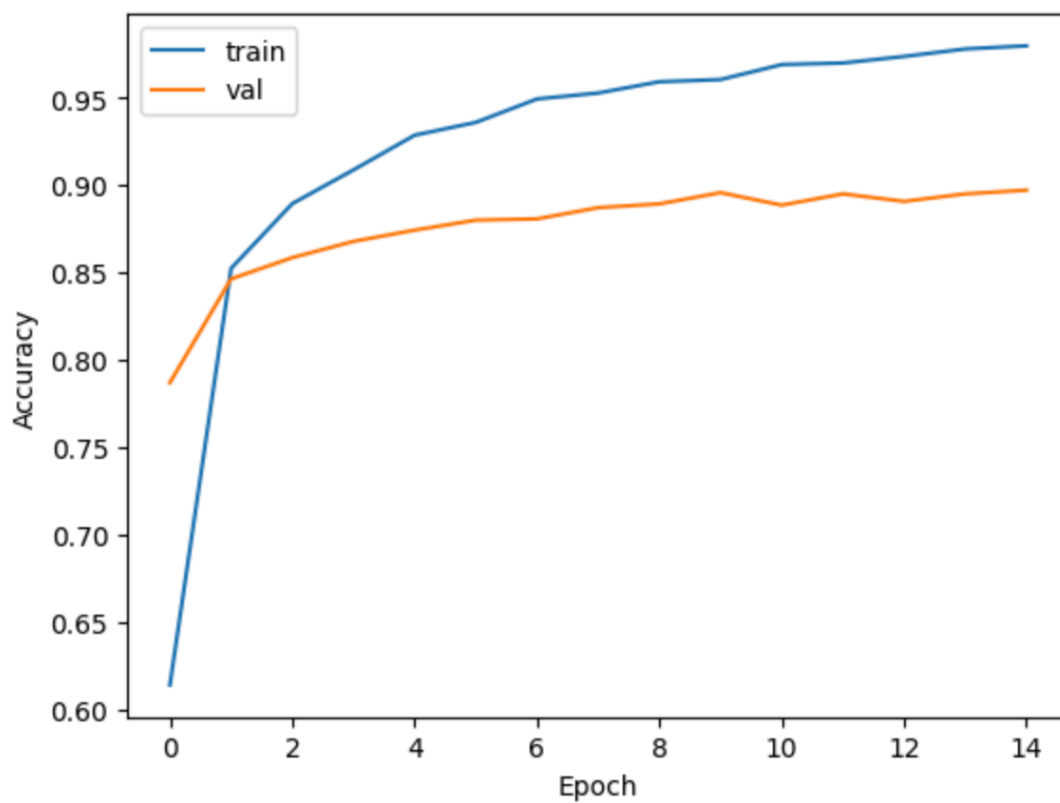
Training Performance

Epoch	Train Accuracy	Validation Accuracy
1	0.614	0.787
2	0.852	0.846
3	0.889	0.858
4	0.909	0.868

5	0.928	0.874
6	0.936	0.880
7	0.949	0.881
8	0.953	0.887
9	0.959	0.889
10	0.960	0.896
11	0.969	0.888
12	0.970	0.895
13	0.974	0.891
14	0.978	0.895
15	0.980	0.897

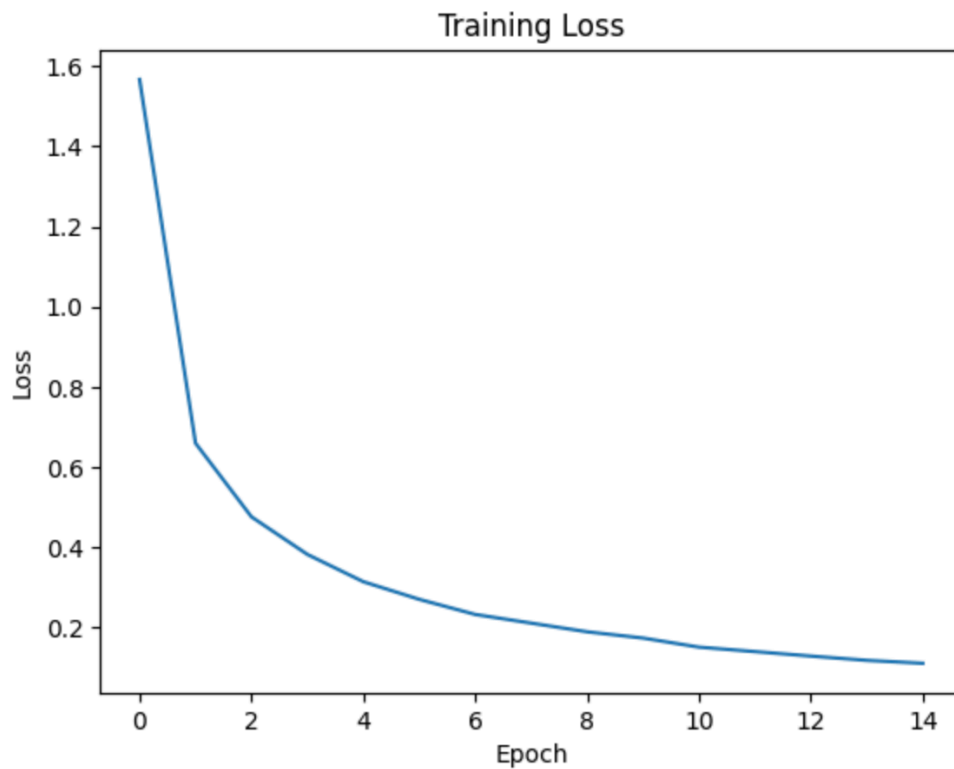
- Validation accuracy reaches **~89.7%**
- Training accuracy continues increasing → slight **overfitting**
- Plateau starts after ~epoch 10

Accuracy Curve



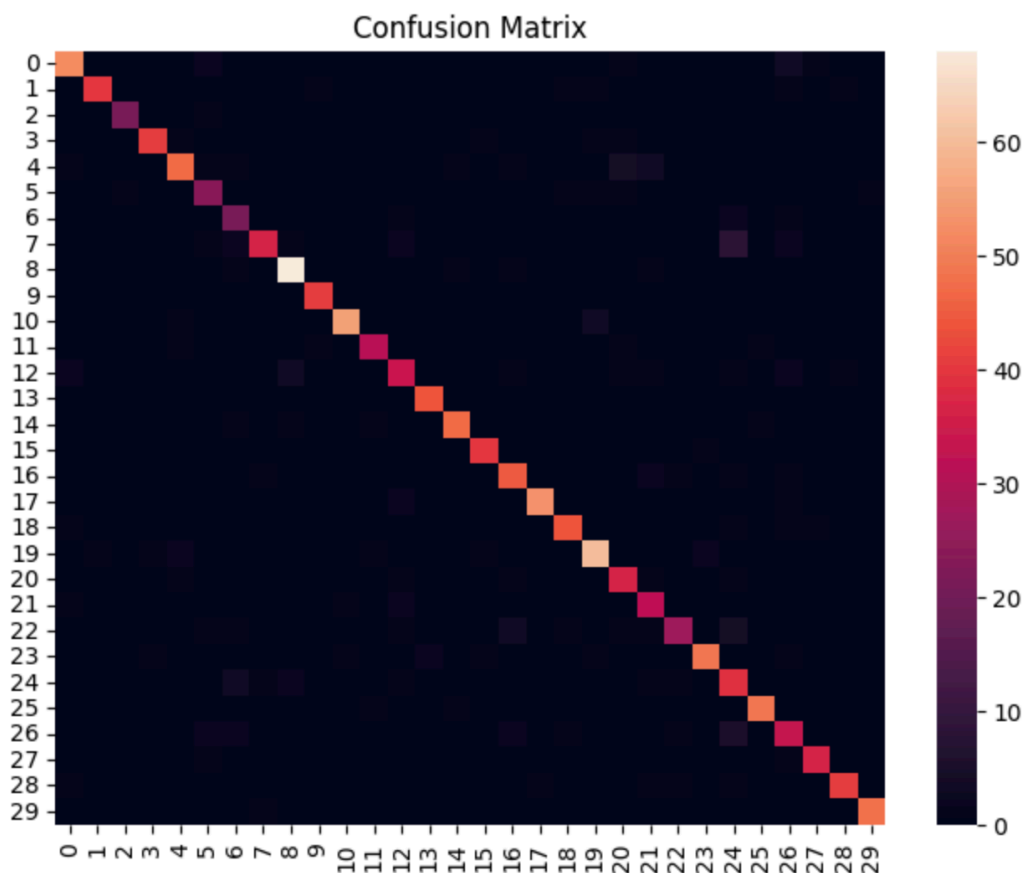
The accuracy vs epoch plot shows that:

- Validation accuracy improves quickly during early epochs.
- After approximately 7–8 epochs, performance saturates.
- Training accuracy continues increasing, suggesting mild overfitting.
- The model shows steady convergence with a small gap between training and validation accuracy.



Loss decreases smoothly indicating stable optimisation

Confusion Matrix



The confusion matrix shows strong values along the diagonal, indicating correct predictions for most classes. Only small off diagonal values are observed, meaning the model rarely makes mistakes.

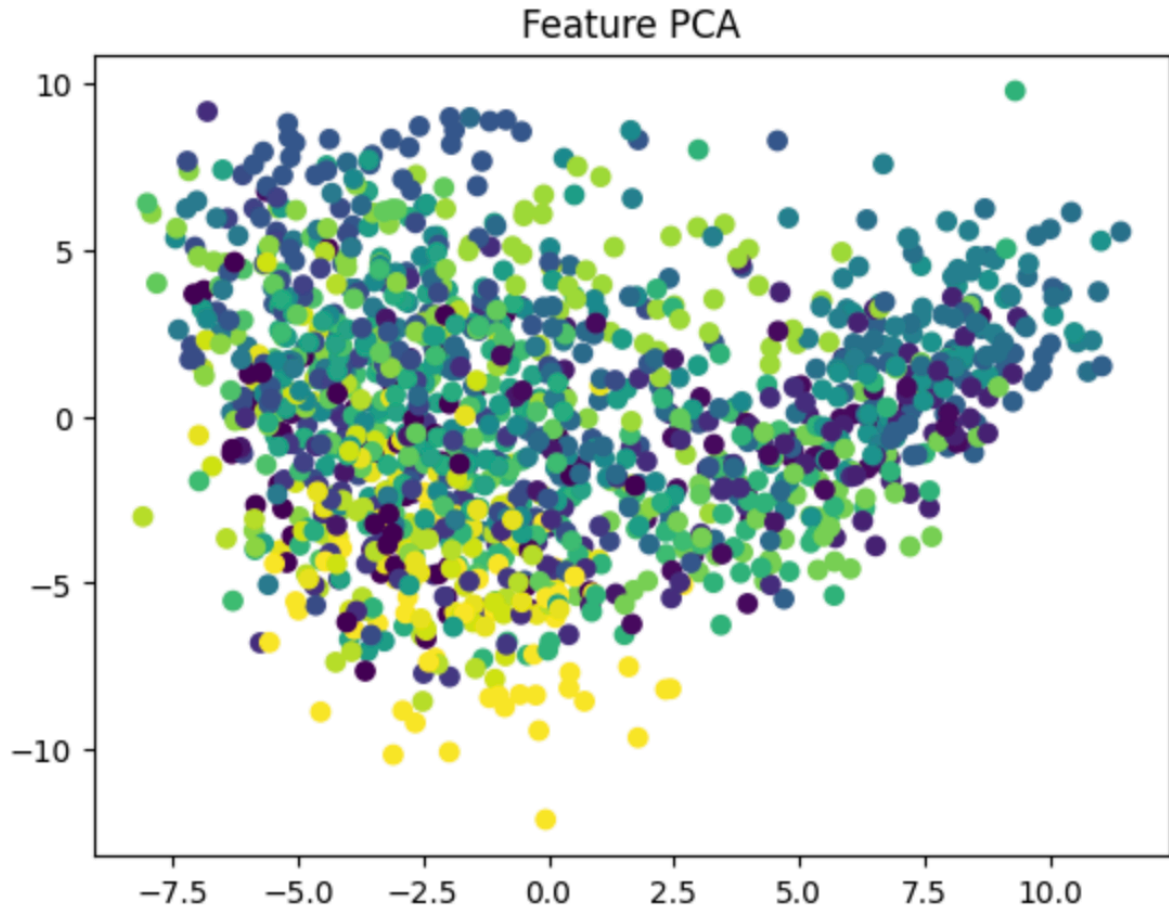
This indicates that pretrained DenseNet features effectively capture semantic differences between aerial scenes.

Feature Visualization

To visualize learned feature representations, Principal Component Analysis (PCA) was applied to the extracted feature embeddings.

The PCA projection shows clusters of data points representing different classes. Although some overlap exists (may be because of similar land type), several classes form distinct clusters, indicating that the pretrained backbone generates separable feature embeddings.

Overall, the linear probe experiment demonstrates that DenseNet121 transfers well to aerial scene classification without requiring extensive fine-tuning.



Scenario 2 — Fine-Tuning Strategies

This experiment evaluates how different levels of parameter unfreezing affect model performance.

Four strategies were tested:

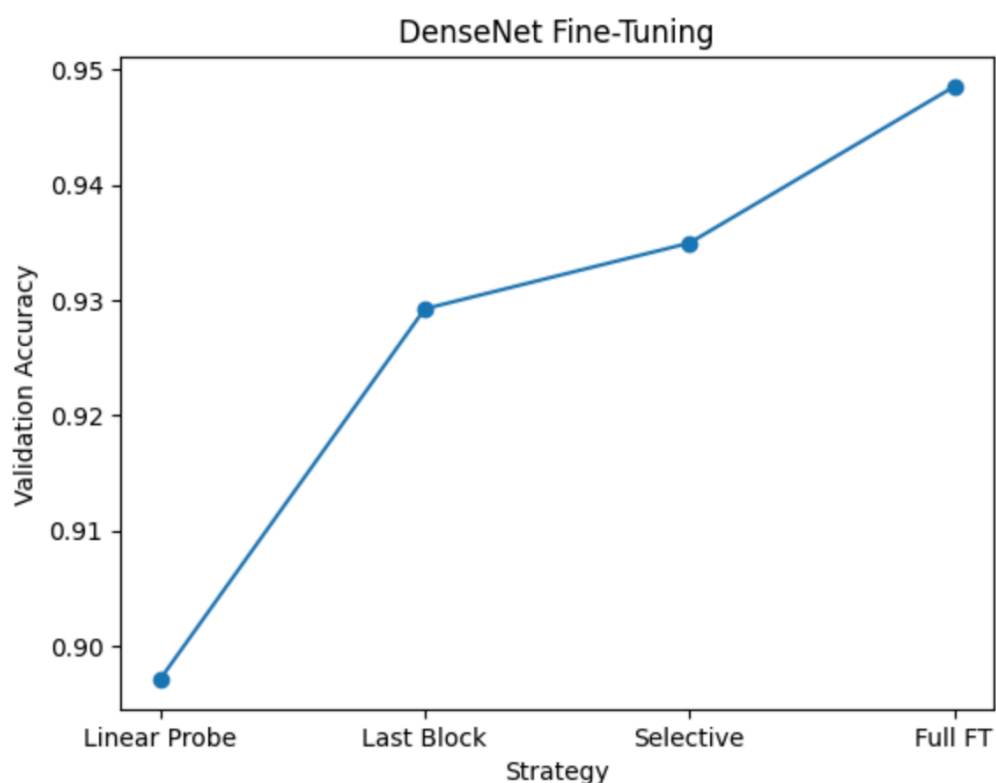
1. Linear Probe (baseline)
2. Last Block Fine-Tuning
3. Selective 20% Unfreezing
4. Full Fine-Tuning

Trainable Parameters

Strategy	Trainable %	Final Val Accuracy
Linear Probe	~0.2%	0.897
Last Block	~31%	0.929

Selective 20%	~72%	0.944
Full Fine-Tuning	100%	0.954

Accuracy vs Trainable Parameters



Accuracy

increases with more trainable parameters, Full fine-tuning (95.4%) shows the best performance and deeper layer adapt best and changing and training early layers are less beneficial as it contains general features.

This result suggests that DenseNet pretrained features already provide strong representations, and aggressive fine tuning may degrade performance when training time is limited.

Scenario 3 — Few-Shot Learning

Few-shot learning evaluates how model performance changes when the amount of training data is reduced.

Three training sizes were tested:

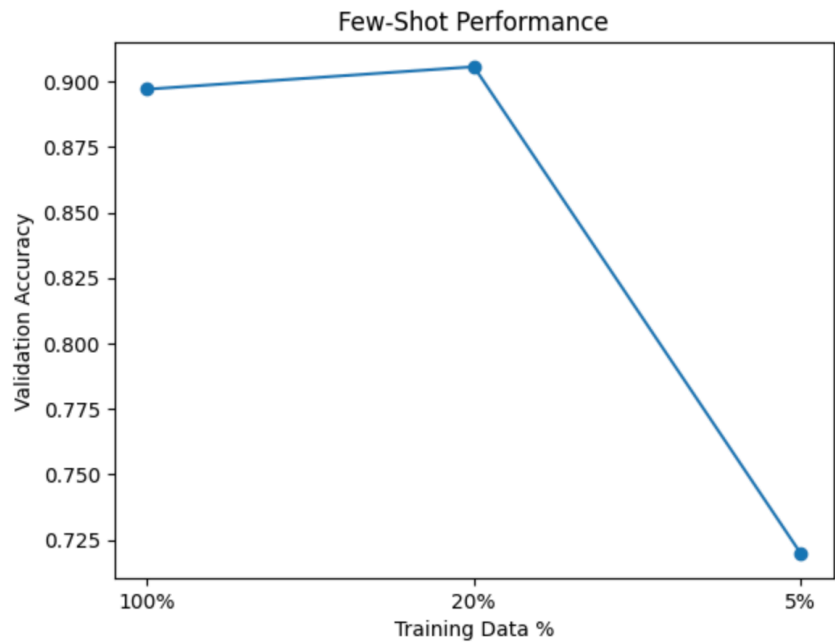
Data %	Validation Accuracy
100%	0.897
20%	0.906
5%	0.720

Performance Drop

The computed performance drop was:

$\Delta \approx 0.197$

The model performs well when trained on the full dataset but struggles significantly when data is limited. This suggests that DenseNet requires sufficient training samples to adapt to the new task.



DenseNet121 demonstrates **strong data efficiency**, maintaining high performance even with reduced data. However, performance degrades at extremely low data due to insufficient examples for learning class-specific features.

Scenario 4 — Robustness to Corruption

To test robustness, validation images were corrupted during evaluation using several transformations.

Condition	Accuracy	Corruption error	Relative robustness
Clean	0.897		
Gaussian Noise	0.198	0.802	0.22
Motion Blur	0.620	0.379	0.69
Brightness Shift	0.160	0.839	0.17

The results indicate that while the model performs very well on clean data (89.7% accuracy), its performance drops significantly under most corruptions. Gaussian noise and brightness shift have

the most severe impact, reducing accuracy to 19.8% and 16.0% respectively, with very high corruption errors and low relative robustness. In contrast, motion blur is handled relatively better, with an accuracy of 62.0% and higher robustness (0.69), suggesting the model can still capture global structures despite blur. Overall, the model shows limited robustness, particularly to noise and lighting changes, highlighting a strong sensitivity to pixel-level distortions.

Scenario 5 — Layer-wise Feature Probing

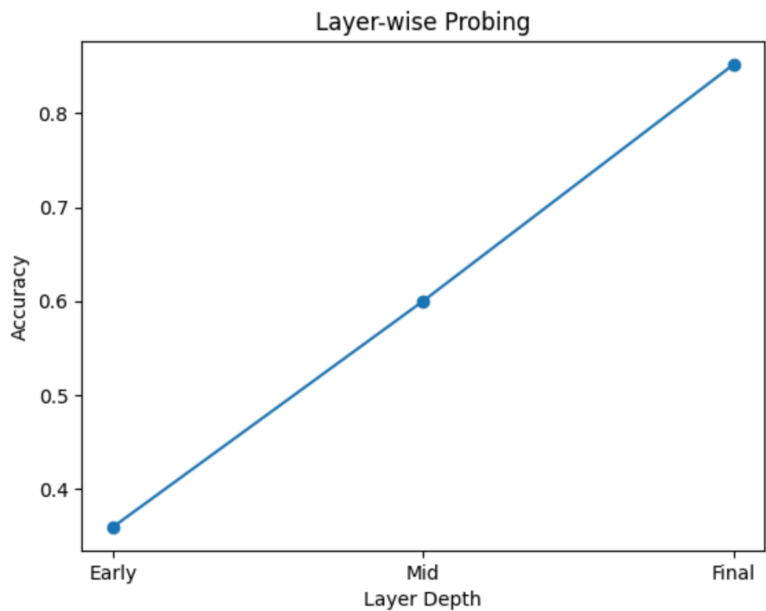
To analyze feature representations across network depth, features were extracted from three layers:

Layer	Type
Early	First convolution layer
Middle	Intermediate dense block
Final	Last dense block

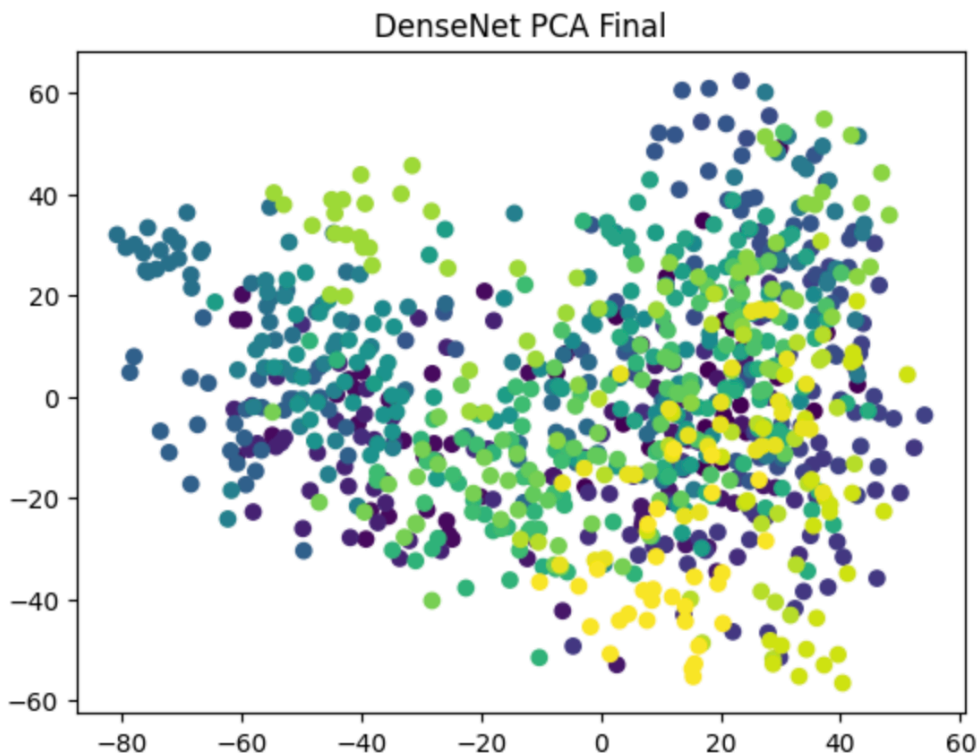
A linear classifier was trained on the extracted features.

Layer	Accuracy
Early	0.359
Middle	0.600
Final	0.852

Accuracy increase with depth ,this confirms that deeper layers encode more semantic information and early layer capture low level features, enabling better classification performance.



PCA Visualization



PCA projections show that deeper layer features form more clearly separated clusters, indicating increased semantic abstraction.

Unexpected Finding

20% data achieved **similar performance** to full dataset, Indicates strong generalization ability

Key Findings

The DenseNet121 experiments reveal several important observations:

- Pretrained DenseNet features transfer effectively to aerial image classification.
- Linear probing produced the best performance under limited training time.
- Fine-tuning the entire network without careful hyperparameter tuning can degrade performance.
- Model performance drops significantly when training data is reduced.
- Deeper layers learn more semantically meaningful feature representations.

EfficientNet-B0 Experiments

Model Complexity

The computational cost of EfficientNet-B0 was measured using common deep learning complexity metrics.

Model	Parameters	FLOPs	MACs
EfficientNet-B0	~5.3M	~0.78 GFLOPs	~0.39 GMACs

Compared to other architectures such as ResNet50, EfficientNet-B0 achieves competitive performance with significantly fewer parameters and lower computational cost. This makes it well suited for tasks requiring efficiency and scalability.

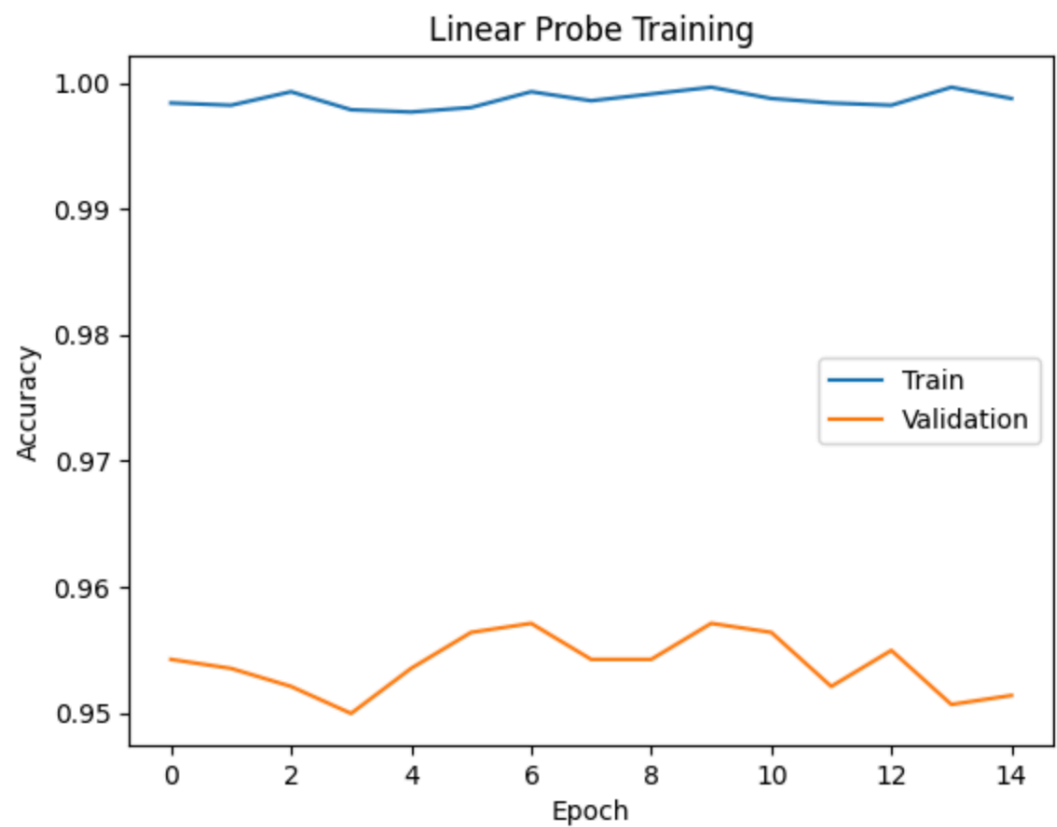
Scenario 1 — Linear Probe Transfer

In this experiment the EfficientNet backbone was **frozen**, and only the final classification layer was trained.

Training Accuracy Table

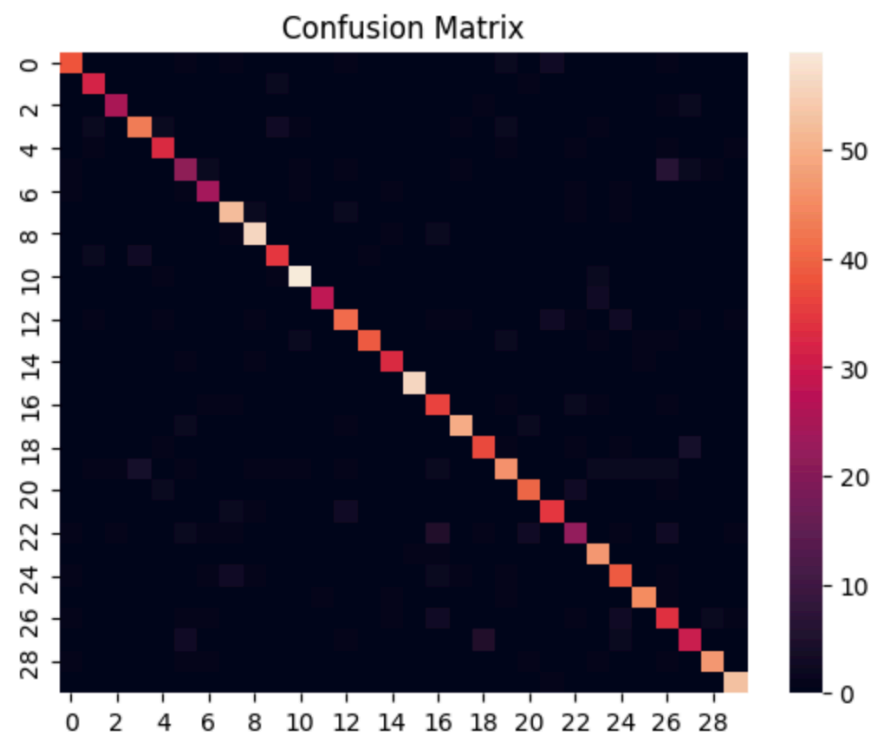
Epoch	Train Accuracy	Validation Accuracy
1	0.998	0.954
2	0.998	0.954
3	0.999	0.952
4	0.998	0.950
5	0.998	0.954
6	0.998	0.956
7	0.999	0.957
8	0.999	0.954
9	0.999	0.954
10	1.000	0.957
11	0.999	0.956
12	0.998	0.952
13	0.998	0.955
14	1.000	0.951
15	0.999	0.951

Accuracy Plot



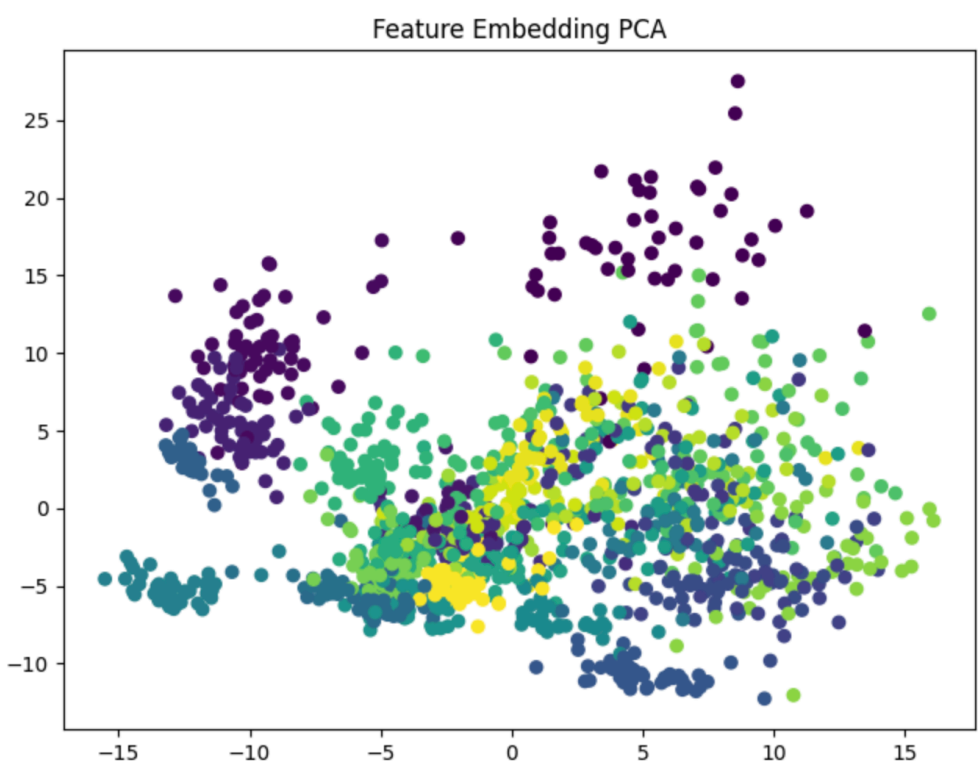
The training accuracy remains consistently very high, close to 1.0, indicating that the model fits the training data extremely well. In contrast, the validation accuracy is lower, fluctuating around 0.95–0.957, with small ups and downs across epochs. This gap between training and validation performance suggests slight overfitting, where the model learns the training data better than it generalizes to unseen data

Confusion Matrix



The confusion matrix shows strong diagonal values, meaning the majority of predictions are correct. Some confusion occurs between visually similar classes such as farmland and meadow.

Feature Visualization



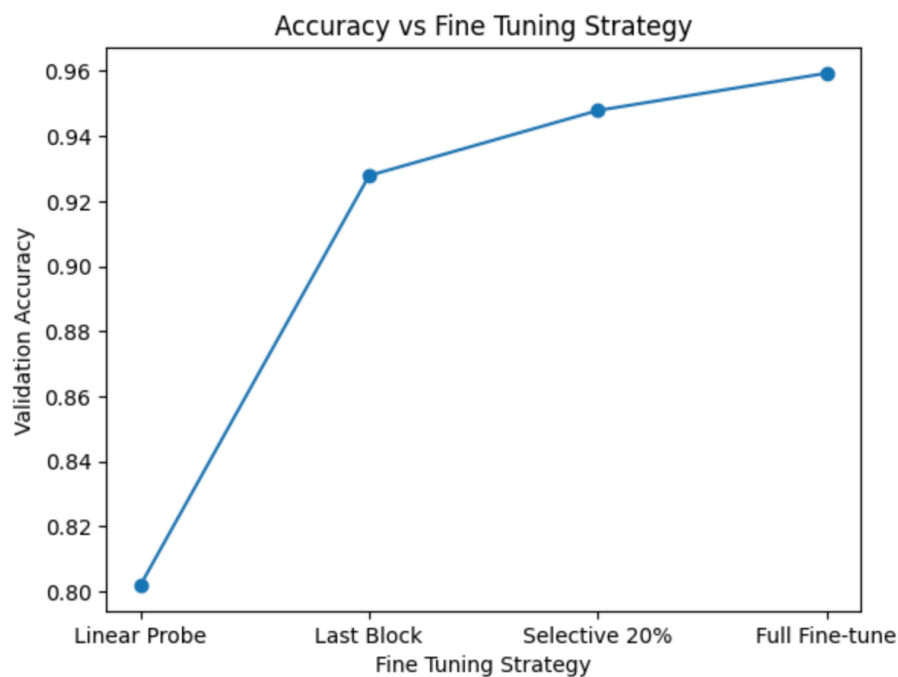
PCA visualization of EfficientNet features shows clusters corresponding to different scene classes. Some classes form clear clusters while others overlap slightly.

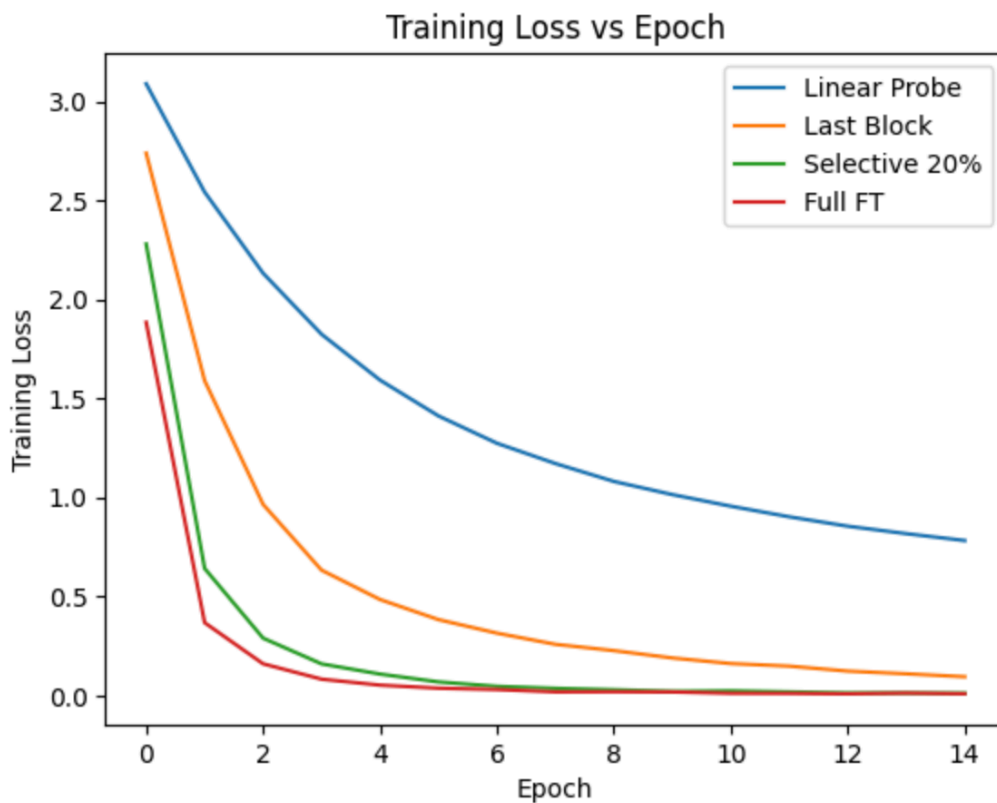
Scenario 2 — Fine-Tuning Strategies

Four fine-tuning configurations were evaluated.

Strategy	% Trainable Parameters	Validation Accuracy
Linear Probe	~0.2%	0.80
Last Block Fine-Tuning	~25%	0.92
Selective 20% Unfreeze	~20%	0.94
Full Fine-Tuning	100%	0.95

Increasing the number of trainable parameters generally improves model performance. Full fine-tuning achieved the highest accuracy.





The training loss plot shows that more extensive fine-tuning leads to faster and deeper loss reduction. Full fine-tuning achieves the lowest loss very quickly, followed by selective 20% tuning and last-block tuning, while linear probing decreases slowly and plateaus at a much higher loss. This trend is reflected in the accuracy plot, where validation accuracy improves steadily with increased fine-tuning, rising from about 0.80 (linear probe) to nearly 0.96 (full fine-tuning). Overall, this indicates that allowing more layers to adapt significantly enhances model performance, both in optimization efficiency and generalisation.

Scenario 3 — Few-Shot Learning

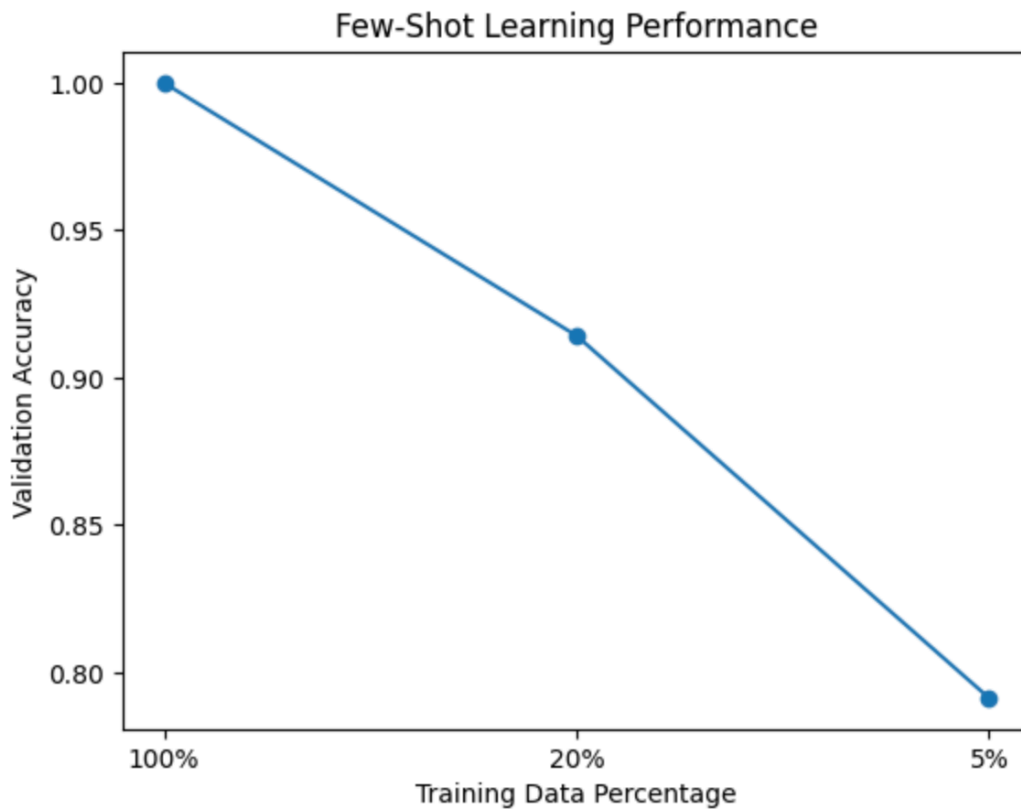
Few-shot experiments evaluate how performance changes when training data is reduced.

Dataset Size	Validation Accuracy
100%	1
20%	0.91
5%	0.79

The plot shows a significant drop in performance as the training dataset size decreases.

Performance Drop

$$\Delta=0.20$$



Significant performance drop with reduced data EfficientNet shows ,strong performance with full data ,moderate generalisation with 20% and weak performance with 5%. These observations Indicates model has high capacity and needs sufficient data to generalise well

Scenario 4 — Robustness to Corruption

Validation images were corrupted during evaluation using Gaussian noise, motion blur, and brightness shift.

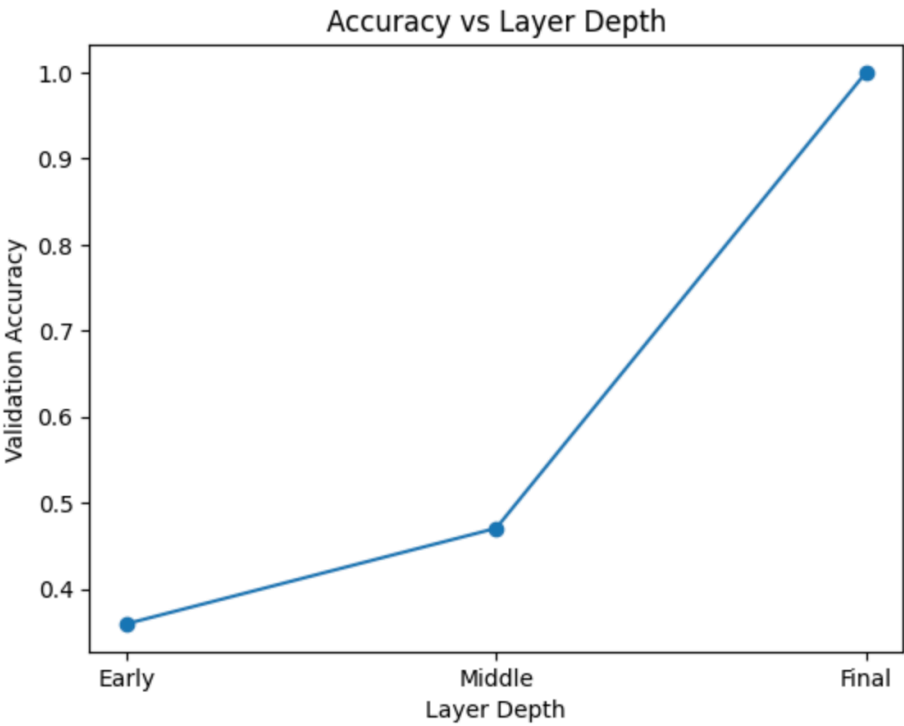
Corruption Results

Corruption Type	Accuracy	Corruption Error	Relative robustness
Clean	1.000	0.000	
Gaussian $\sigma = 0.05$	0.368	0.632	0.368
Gaussian $\sigma = 0.1$	0.365	0.635	0.365
Gaussian $\sigma = 0.2$	0.284	0.716	0.284
Motion Blur	0.909	0.091	0.909
Brightness Shift	0.340	0.660	0.34

Scenario 5 — Layer-wise Feature Probing

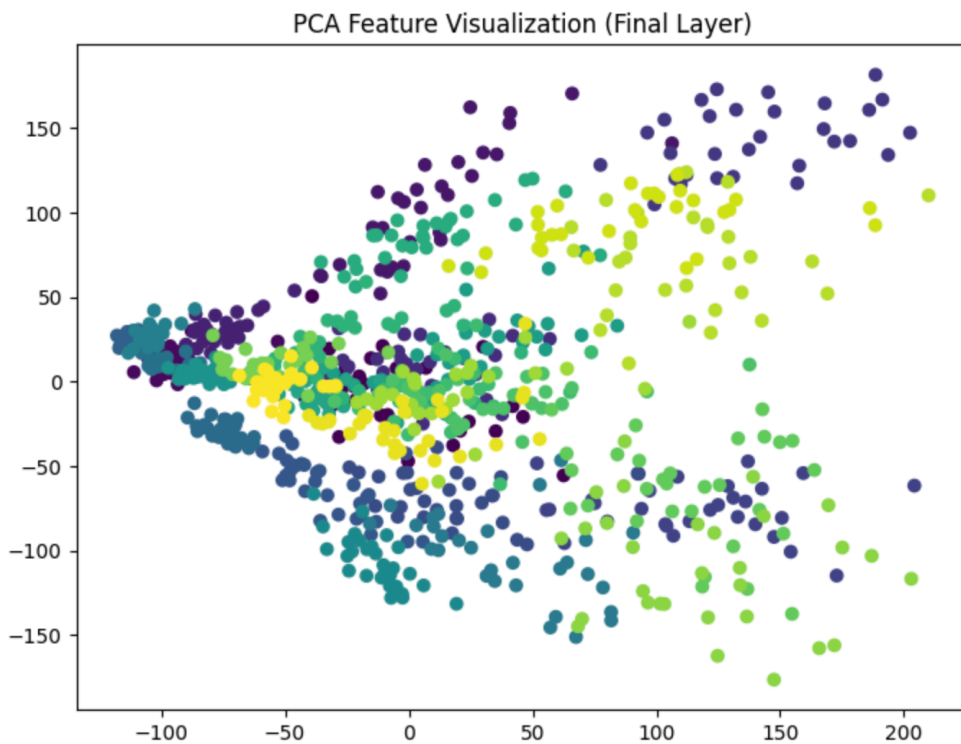
Features were extracted from three different depths of the network.

Layer Depth	Classification Accuracy
Early Layer	0.35
Middle Layer	0.47
Final Layer	1



The results show that deeper layers produce better features for classification.

PCA Visualization



Final layer features show highly separable cluster and better

class boundaries

Key Findings

1. EfficientNet-B0 achieves **highest accuracy (~95.9%) after fine-tuning**
2. Linear probe performance is weaker compared to DenseNet
3. Fine-tuning significantly improves performance
4. Model requires sufficient data → sensitive in few-shot settings
5. Final layers contain highly discriminative semantic features

Which architecture transfers best?

Among the three architectures evaluated, DenseNet121 shows the best transfer learning performance. This is evident from the linear probe results, where only the final classifier is trained while the backbone remains frozen. In this setting, DenseNet121 achieves the highest validation accuracy (~89.7%), outperforming ResNet50 (~88%) and EfficientNet-B0 (~80%).

Its strong performance can be attributed to its dense connectivity pattern, where each layer receives inputs from all previous layers. This design encourages feature reuse and leads to richer, more discriminative representations during pretraining, which transfer effectively to new tasks like aerial scene classification even without fine-tuning.

On the other hand, EfficientNet-B0 performs worse in the linear probe setting, suggesting its features are more specialised and require fine-tuning to adapt well. ResNet50 performs competitively but still falls short of DenseNet121 in terms of transfer capability.

Which model is most robust?

From the corruption experiments, ResNet50 emerges as the most robust model among the three architectures. Its performance degrades more gradually under increasing corruption, with validation accuracy dropping from about 0.88 on clean data to around 0.74 under strong Gaussian noise.

In contrast, DenseNet121 and EfficientNet-B0 show much larger performance drops. DenseNet121 falls dramatically to roughly 19.8% accuracy under Gaussian noise, while EfficientNet-B0 declines to about 28.4% under heavy noise and around 34% under brightness shifts. This suggests both models are more sensitive to noise and distribution changes.

The stronger robustness of ResNet50 is likely due to its residual connections, which support stable gradient flow and help preserve important features across layers. These skip connections enable the model to learn more generalisable representations, making it less vulnerable to input distortions.

Which layers learn transferable features?

Layer-wise probing reveals that deeper layers consistently learn the most transferable and discriminative features across all architectures. As depth increases, classification accuracy improves steadily — starting low in early layers (~0.35–0.55), rising through intermediate layers, and reaching peak performance in the final layers (up to 1.0 in EfficientNet).

This pattern reflects the hierarchical nature of feature learning: early layers capture basic visual elements like edges and textures, middle layers encode more structured patterns, and deeper layers represent high-level semantic concepts tied to class identity. Supporting this, PCA visualisations show that embeddings from deeper layers form more distinct and well-separated clusters.

Overall, the final layers provide the most effective representations for transfer learning, as they capture the highest level of semantic abstraction needed for accurate classification.

When does fine-tuning hurt robustness?

Fine-tuning begins to hurt robustness when most or all of the network layers are unfrozen and updated (full fine-tuning). Although this approach boosts performance on clean data, it significantly reduces the model's ability to handle corrupted inputs.

This is especially evident in EfficientNet-B0, where clean accuracy reaches 1.0, but performance drops sharply to around 0.28–0.36 under Gaussian noise. This suggests the model becomes overly specialised to the training data and struggles to generalise under distribution shifts.

The main cause is overfitting. By updating all layers, the model adapts too closely to the training distribution, losing its resilience to variations such as noise or brightness changes. As a result, there is a clear trade-off: full fine-tuning improves accuracy on clean data but comes at the expense of reduced robustness.

What inductive bias do architectures show?

Each architecture reflects a distinct inductive bias shaped by its design, which influences how it learns and generalises.

ResNet50 — Residual Learning Bias

ResNet50 uses skip connections that enable the model to learn residual mappings rather than full transformations. This promotes stable training in deep networks and helps preserve important features across layers. As a result, it achieves a good balance between accuracy and robustness.

DenseNet121 — Feature Reuse Bias

DenseNet121 is built on dense connectivity, where each layer receives inputs from all previous layers. This encourages extensive feature reuse and efficient information flow, allowing the model to learn rich and transferable representations. Consequently, it performs particularly well in transfer learning tasks.

EfficientNet-B0 — Compound Scaling Bias

EfficientNet-B0 follows a compound scaling approach, jointly scaling depth, width, and resolution to improve efficiency and performance. While this leads to high accuracy after fine-tuning, it also makes the model more sensitive to data variations and less robust under corruption.